AutoIA Path Forward: V1 Exit Gate and V2 Research/Engineering Plan

Planning baseline: AutoIA `1.0.2`
Scope: what should happen after the V1 semantic surface is frozen, without weakening the existing proof contract.

---

1. Executive direction

AutoIA V2 should not be defined as "support more Python." It should be defined as **extending the set of common engineering computations for which AutoIA has mathematically explicit, dependency-aware, fail-closed certification semantics**.

The V1 architecture has established five principles that should control V2:

1. ordinary numerical libraries may provide proposals, but AutoIA independently verifies proof obligations;
2. dependence on original physical inputs should be preserved until there is a compelling reason to intervalize;
3. uncertain control flow and algorithmic domains are handled as physical/path proof problems, not by clipping;
4. optional controllers may affect cost and sharpness but never soundness;
5. application-specific mathematics remains outside AutoIA behind the public application-theorem interface.

The permanent application dependency direction remains

application package (for example downstream GP application) $\rightarrow$ AutoIA.

AutoIA V2 should not absorb GP, Bayesian-optimization, CALPHAD, CFD, or other application-specific theory simply because those applications use AutoIA.

---

2. V1 exit gate comes before V2 development

Before adding V2 semantics, freeze a V1 release candidate and complete an adversarial validation campaign.

Required V1 exit criteria

- no known incorrect certified containment;
- no silent fallback from rigorous to nominal arithmetic;
- no ambiguity in `machine_rigorous` meaning;

- clean install from a wheel without the source tree;
- complete numbered example suite passes from the wheel;
- trusted arithmetic/high-precision audit passes;
- external engineering corpus contains multiple independently authored codes;
- unsupported common operations fail explicitly and intelligibly;
- shrinking-box behavior is sane for supported representative examples;
- documentation and code support matrix agree;
- dependent application packages pass against the clean wheel;
- outstanding monolithic-test-process stability issue is understood or eliminated.

V2 feature work should begin only after the V1 contract has a stable tag/branch. Otherwise new research work will continually move the release boundary.

---

3. V2 priority A: symmetric and generalized-symmetric eigenvalues

This is the clearest next semantic family exposed by the adversarial engineering campaign.

3.1 Target problem

Support

$$A(p)v_i(p) = \lambda_i(p)v_i(p)$$

for real symmetric `A(p)`, and

$$K(p)v_i(p) = \lambda_i(p)M(p)v_i(p)$$

for real symmetric `K(p)` and symmetric positive-definite `M(p)`. Here:

- `p` is the vector of uncertain physical inputs;
- `lambda_i(p)` is an ordered eigenvalue;
- `v_i(p)` is a corresponding eigenvector.

3.2 V2.0 target should be eigenvalue-only

Do **not** make general eigenvectors a prerequisite for useful `eigh` support. Many engineering calculations use only frequencies, stability margins, buckling factors, or spectral bounds.

Initial semantics should certify eigenvalue intervals and preserve mode clusters when separation cannot be proved.

If certified intervals satisfy

$$\overline{\lambda}_i < \underline{\lambda}_{i+1},$$

then modes `i` and `i+1` are spectrally separated over the box. If intervals overlap, AutoIA should report a certified cluster rather than inventing a persistent identity.

3.3 Proposal/proof architecture

SciPy/LAPACK may produce nominal eigenpairs at one or more reference points. These are proposals only. Candidate certification routes to research include:

- residual-based eigenvalue inclusion theorems;
- generalized Rayleigh quotient bounds;
- verified transformation using positive-definite mass matrices;
- interval/Krawczyk verification of isolated eigenpairs;
- min-max/Courant-Fischer variational bounds;
- spectral cluster certification.

The proof route should be chosen by theorem strength and dependency preservation, not by closeness to LAPACK internals.

3.4 Eigenvectors later and conditional

Individual eigenvectors are not well-defined at repeated eigenvalues and can rotate rapidly near clustered modes. V2 should certify eigenvectors only when a spectral-gap theorem establishes an isolated eigenspace. For repeated/clustered eigenvalues, certify invariant subspaces or decline eigenvector certification.

3.5 Acceptance corpus

- 2x2 and 3x3 analytic matrices with known eigenvalues;
- uncertain diagonal and coupled stiffness matrices;
- generalized mass/stiffness examples;
- mode crossing example;
- nearly repeated eigenvalue example;
- EXUDYN-style natural-frequency calculation;
- finite-element modal example from an independently authored package.

---

4. V2 priority B: validated ODE initial-value problems

`scipy.integrate.solve_ivp` is one of the most important common engineering boundaries not covered by V1.

4.1 Target problem

For state `y(t,p)` satisfying

$$\frac{dy}{dt} = f(t, y, p), \qquad y(t_0, p) = y_0(p),$$

certify `y(t,p)` over uncertain parameters `p` and, eventually, uncertain initial conditions.

4.2 Do not certify an adaptive RK trace by replaying floats

The V2 semantic should treat ordinary numerical integration as a proposal. A rigorous route should use a validated numerical-analysis theorem, for example:

- interval Taylor methods;
- validated Runge-Kutta defect/residual bounds;
- Picard-Lindelof contraction on time slabs;
- a posteriori enclosure around a nominal trajectory;
- polynomial/Taylor models in physical parameters with rigorous time remainder.

4.3 Staged scope

V2.0: smooth nonstiff IVPs, fixed output times, no events.
V2.1: events with certified root/event-time semantics.
V2.2: selected stiff systems/implicit methods.
Later: DAE index-specific semantics.

The first implementation should prefer a small, explicit theorem-backed subset over claiming all `solve_ivp` methods.

4.4 Dependency challenge

Returning an independent interval state at every time step may create catastrophic wrapping. V2 should investigate parameter-dependent polynomial/generator trajectory representations and recentering strategies rather than adopting naive interval time marching.

---

5. V2 priority C: semantic nonlinear optimization, carefully scoped

Engineering codes often call `minimize`, `least_squares`, or constrained optimizers. General global optimization is too broad to become one generic semantic primitive.

A useful V2 distinction is:

5.1 Optimization as an internal state solve

If the code uses an optimizer to solve first-order stationarity equations for a locally isolated optimum, AutoIA may be able to verify KKT conditions and second-order sufficient conditions around a nominal proposal.

5.2 Optimization as a global value operation

If downstream code relies on

$$g(p) = \min_{x \in X} f(x, p),$$

then certification requires rigorous lower and upper value bounds over the decision set `X`. This is a global optimization problem and should use a theorem-backed branch-and-bound/application layer, not a local optimizer certificate.

5.3 Proposed V2 contract

- recognize common SciPy optimizers;
- distinguish local stationary-point semantics from global optimum semantics;
- never infer globality merely because SciPy reports success;
- expose clean unsupported diagnostics until a theorem-backed subset exists.

---

6. V2 priority D: richer interpolation and tabular models

V1 supports fixed-data linear `RegularGridInterpolator`.

Natural extensions are:

6.1 Nearest-neighbor

Mathematically simple but discontinuous. It should use explicit Voronoi/cell path splitting, not a continuous enclosure approximation.

6.2 Piecewise polynomial splines

PCHIP/cubic/quintic semantics should be added only after the actual fixed-data piecewise polynomial coefficients can be obtained or reconstructed rigorously and their interval/domain behavior is understood.

6.3 Uncertain table data

An important research direction is uncertainty in table values themselves. This should preserve shared uncertainty structure instead of assigning independent

intervals to every table entry when values derive from common parameters or calibration errors.

6.4 Scattered-data interpolation

This is much harder because cell/triangulation topology may itself depend on data. Do not group it casually with rectilinear-grid interpolation.

---

7. V2 priority E: stronger dependency representations

V1's polynomial and shared-generator representations provide a strong foundation, but difficult repeated nonlinear operations can still accumulate remainder.

Research directions:

- generator compression with certified error accounting;
- richer products/interactions among auxiliary generators;
- affine-arithmetic-like noise symbols coupled to polynomial retained structure;
- sparse polynomial bases selected by structure rather than fixed total degree alone;
- automatic recentering/re-expansion at strategic program points;
- component/group representations for large vector states;
- certified low-rank structure for matrix-valued uncertainty.

Any compression or basis selection must produce an explicit remainder enclosure. Heuristic dimension reduction without certified discarded-error bounds is not acceptable.

---

8. V2 priority F: constrained and nonrectangular physical domains

V1's natural physical domain is a Cartesian interval box. Many engineering inputs satisfy constraints such as

$$x_i \geq 0, \qquad \sum_i x_i = 1$$

for compositions, or affine conservation constraints.

Possible V2 paths:

1. exact affine parameterization of lower-dimensional domains;
2. simplex-specific Bernstein/ranging methods;
3. polytope-aware monotonicity and branch proofs;

4. domain objects carrying certified affine constraints;
5. constraint-aware subdivision that never wastes work outside the physical set.

A good first step is exact affine reparameterization, because it preserves the V1 box machinery internally while removing one dependent coordinate.

---

9. V2 priority G: large-scale many-box and heterogeneous acceleration

V1 already includes two important foundations: multicore CPU many-box proof and optional PyTorch MPS/CUDA proposal acceleration with independent CPU verification. V2 should improve throughput without weakening that trust boundary.

9.1 Batch-native CPU proof kernels

Move beyond process-level parallelism where profiling justifies it: packed leading-box dimensions, lower-allocation polynomial/range kernels, faster cache indexing, and batched structured-matrix theorems. The V1 process backend remains the portability baseline.

9.2 Richer GPU proposal/assist workloads

Extend the V1 MPS/CUDA proposal interface to application-owned tasks such as candidate split scoring, nominal matrix preconditioners, root/eigenpair/trajectory proposals, and batched sampling diagnostics. GPU outputs remain explicitly non-rigorous until independently verified.

9.3 GPU-assisted proof with CPU verification

A GPU may compute candidate bounds, matrix summaries, or compressed structures, while trusted CPU arithmetic verifies the final inequalities before any certificate is accepted. This can reduce CPU work without expanding the trusted hardware base.

9.4 Fully trusted CUDA proof later

A true trusted CUDA path requires explicit directed-rounding semantics, compiler/toolchain constraints, cross-architecture tests, and independent audit. Apple MPS/Metal should remain proposal-side until equivalent rigorous primitives exist. A CUDA implementation should be tested on more than one NVIDIA architecture before being called portable. Generic tensor execution is not automatically rigorous because the hardware is fast.

---

10. V2 priority H: language/runtime coverage

Expand only when a clear engineering need and proof semantic exist.

Candidates:

- more NumPy reductions and broadcasting patterns;
- selected matrix factorizations;
- fixed-size loops/iterative algorithms whose semantics can be certified;
- dataclass/structured state containers;
- selected SciPy special functions;
- event/root interfaces built on existing implicit-root semantics;
- controlled support for source-backed callables with local imports.

Avoid a blanket goal of "run arbitrary Python." Dynamic code generation, compiled extension internals, opaque C/Fortran state, I/O, random external state, and side effects may remain outside the certification language.

---

11. V2 priority I: theorem/plugin ecosystem

The application theorem interface should mature into a stable extension ecosystem.

Needed improvements:

- richer theorem metadata and provenance;
- machine-readable statement of assumptions;
- theorem composition diagnostics;
- reusable certified matrix/vector operations;
- optional theorem registry owned by application packages, not AutoIA core;
- serialization of certificate summaries without serializing trusted internal objects;
- reproducibility metadata for external model snapshots.

downstream GP application remains the reference example of this architecture: it owns GP and acquisition-function theorems and uses AutoIA only for generic proof mechanics.

---

12. V2 priority J: certificate explainability and audit artifacts

V1 has useful diagnostics, but V2 can make certificates more reviewable without making them heavier by default.

Potential features:

- concise proof tree showing which operations required subdivision;

- domain obligation ledger;
- branch-surface summary;
- representation/remainder ancestry summary;
- trusted-kernel environment fingerprint;
- optional JSON certificate report;
- reproducible command line for replay;
- comparison of candidate proof routes and which interval intersection tightened the result.

The certificate report must distinguish mathematical proof facts from controller decisions and diagnostic samples.

---

13. Research questions that should remain explicit

V2 should maintain a written list of unresolved research questions rather than burying them in implementation choices.

1. When should a dependency-bearing representation be recentered automatically?
2. How should shared generators be compressed with the best cost/sharpness tradeoff while preserving proof?
3. What is the best verified eigenvalue method for parameter-dependent generalized symmetric problems?
4. How can ODE parameter dependence be preserved without classical wrapping explosion?
5. Which semantic primitives deserve direct AutoIA support versus the external application-theorem interface?
6. How should automatic configuration use structural signals without becoming application-tuned?
7. How should proof-driven subdivision allocate work across path, domain, and output-tightening objectives?
8. Can a portable fully trusted GPU backend be made practical without expanding the trusted code base excessively?
9. How should mode/eigenspace identity be represented when eigenvalues cross?
10. What certificate information is most useful to an engineering reviewer without overwhelming normal users?

---

14. Benchmark and validation program for V2

Each new semantic family should have four levels of evidence.

Level 1: analytic microproblems

Known exact solutions and adversarial boundary cases.

Level 2: independently computed high-precision audits

Dense or randomized points checked with arbitrary precision or independent solvers.

Level 3: external engineering functions

Code authored independently of AutoIA and kept as unchanged as practical.

Level 4: realistic coupled workflows

Several supported semantics interacting: branches + interpolation + implicit solve + linear algebra, for example.

Every benchmark should record:

- certified containment;
- width/sharpness relative to diagnostic sampling or known exact range;
- shrinking-box behavior;
- runtime;
- path/subdivision count;
- failure type if not certified;
- representation/configuration used;
- whether the operation was direct proof or proposal-plus-verification.

---

15. Proposed V2 milestone sequence

V2-M0: freeze and ship V1 - COMPLETE

AutoIA 1.0.2 preserves the frozen V1 validated-computation substrate with CPU binary64 proof trust, conservative safe-auto multicore scheduling for finite many-box work, and explicit opt-in MPS/CUDA proposal assist. Downstream downstream package needs should use the public AutoIA boundary rather than reopening V1 with optimizer-specific semantics.

V2-M1: validated symmetric eigenvalues

`scipy.linalg.eigh` eigenvalue-only subset, spectral separation/clustering, EXUDYN-style frequency acceptance test.

V2-M2: validated nonstiff ODE prototype

Small smooth `solve_ivp`-style subset with fixed output times and rigorous a posteriori trajectory enclosure.

V2-M3: dependency and scaling improvements

Generator compression/recentering, many-box performance, certificate diagnostics.

V2-M4: richer tabular models and constrained domains

Nearest/piecewise-polynomial interpolation as justified; affine/simplex domain support.

V2-M5: selected optimization semantics

Local KKT verification and/or theorem-backed global value primitives, with explicit distinction between them.

V2-M6: trusted heterogeneous proof research

Build on the V1 proposal/CPU-verification interface. Investigate audited CUDA directed-rounding kernels and cross-architecture validation; fully trusted GPU arithmetic remains a separate research gate rather than an assumption.

The exact numbering can change. The important ordering is that each semantic family earns its way into the product through a proof contract and external acceptance case.

---

16. What should explicitly not be a V2 goal

- supporting arbitrary Python by coercing AutoIA values to floats;
- certifying third-party compiled libraries whose semantics AutoIA cannot inspect or verify;
- adding application-specific equations to the core because one benchmark needs them;
- treating nominal solver convergence as a proof;
- hiding mode crossings, branch ambiguity, or singularities behind numerical tolerances;
- using machine-learning models to decide proof truth;
- making GPU execution synonymous with rigorous arithmetic;
- increasing polynomial order/subdivision indefinitely as a substitute for understanding a general semantic gap.

---

17. Decision rule for adding a V2 capability

A proposed capability should enter AutoIA core only if all of the following are true:

1. it represents a broadly reusable computational semantic rather than one application's theorem;
2. its mathematical contract can be stated independently of a specific numerical implementation;
3. there is a rigorous verification/enclosure route;
4. failure can be detected and reported closed;
5. interaction with original-input dependence is understood;
6. it has at least one independently authored engineering acceptance case;
7. it can be regression-tested without relying only on nominal samples;
8. its trusted-code expansion is proportionate to its value.

Otherwise, prefer an external application theorem/plugin or an explicit unsupported boundary.

---

18. Immediate next actions after AutoIA 1.0.2

1. Treat the AutoIA 1.x architecture as frozen for downstream certified-optimization work unless a genuinely application-independent correctness or specification defect is found; 1.0.2 is one such backward-compatible specification hardening patch.
2. Build downstream package as a separate package that depends on the public AutoIA API.
3. Place downstream GP application and downstream multi-output application under downstream package as optimization/application layers, not inside AutoIA.
4. Keep AutoIA V2 research on a separate development line, beginning with general capabilities such as validated symmetric eigenvalues rather than downstream package-specific formulas.
5. Require any future AutoIA-core addition to satisfy the general capability decision rule above and to bring an independent engineering acceptance case.

This preserves the product discipline established during V1: **external engineering evidence first, general mathematics second, implementation third, and only then a new support claim.**

1.0.2 lesson: semantic recognition beats user adapters when the boundary is narrow

The fitted-GP work provides a general architectural lesson for future AutoIA semantics. When a common library object has a compact, mathematically explicit operation that can be reconstructed from fixed stored numerical state, AutoIA should prefer local semantic recognition over forcing users to rewrite ordinary source into a special adapter language. This is the same usability principle already used for supported linear and nonlinear solves.

The condition is strict: the recognized boundary must remain narrow, mathematically specified, independently testable, and fail-closed. AutoIA 1.0.2 does not attempt arbitrary Torch execution. It recognizes only the supported query construction and exact-GP posterior surface, freezes fixed numerical state, and evaluates the proof-critical uncertain-query formulas with the trusted backend. Future integrations should satisfy the same standard before becoming transparent public semantics.